ACADEMIC PROJECT REPORT & COMPREHENSIVE DEFENSE MASTERCLASS

# LegalLens: Whole-Judgment NLP Classifier for Indian Court Judgments

## Empirical Topic Classification (Civil vs. Criminal) on Indian Kanoon Judicial Narratives with Complete Software Architecture & Explainable AI

A comprehensive technical manual and speaking guide designed for software teammates, academic evaluators, and viva faculty examiners.

PROJECT AUTHORS & RESEARCH INVESTIGATORS

**Rishi Srivastava**
Registration No: **23FE10CDS00225**
Specialization: Computer Science & Data Science

**Kunwar Suryansh**
Registration No: **23FE10CDS00220**
Specialization: Computer Science & Data Science

| PRIMARY BENCHMARK | CHAMPION ARCHITECTURE | TEST SET MACRO F1 |
|---|---|---|
| **Jud-IPL (CloLex 2024)** | **Balanced Linear SVM (LinearSVC)** | **0.906 (93.6% Accuracy)** |
| **INFERENCE LATENCY** | **AUTOMATED UNIT TESTS** | **LIVE CLOUD APPLICATION** |
| **15 ms (2.05 MB Model)** | **28 / 28 Passed (100%)** | **legallens.streamlit.app** |

Repository: **https://github.com/Mercer18/LegalLens** | Production Release v1.0.0 | Department of Computer Science & Engineering

# Executive Abstract & Introductory Speaking Masterclass

> 🎙️ **How to Read and Speak About This Document (For Teammates & Faculty)**
>
> *"Welcome to LegalLens. If you are a teammate joining our development team or an academic examiner evaluating our thesis, this report is structured to walk you through every design decision as if we are sitting together at a whiteboard. We will explain not only **what** code and models were built, but **why** alternative paths (such as standard BERT transformers or metadata shortcuts) were deliberately rejected in favor of an optimized, high-speed, mathematically explainable machine learning pipeline. Every formula, data cleaning step, backend script, and frontend widget has an empirical reason for existing."*

**LegalLens** is an end-to-end applied Natural Language Processing (NLP) system developed to classify raw, whole-text Indian judicial opinions into the two foundational pillars of the Indian judicial hierarchy: **Civil Law** and **Criminal Law**.

Unlike commercial legal search engines that rely on procedural metadata—such as formal petition codes (e.g., "*CRL.A.*" vs. "*C.A.*"), court names, bench compositions, or outcome tags—LegalLens operates under a **strict text-only boundary**. It ingests the uncurated, multi-page judicial narrative directly, extracting discriminatory legal n-grams, statutory citations, and procedural collocations to determine jurisdictional category.

### Core Research & Engineering Achievements:

- **Systematic Data Audit & Lineage Resolution:** Acquired 53,446 raw judicial cases from Indian Kanoon. Executed an empirical 5-stage cleaning waterfall that resolved missing labels, contradictory clusters, and pruned 7,653 exact duplicate re-uploads, establishing a pristine modeling corpus of **33,717 unique court judgments**.
- **Cryptographic Leakage Protection:** Implemented a strict 80/10/10 stratified partition (26,973 Train / 3,372 Val / 3,372 Frozen Test) validated via pairwise SHA-256 text hash intersections, ensuring **0.0% data contamination** across all splits.
- **High-Speed, Full-Document NLP Pipeline:** Designed a 40,000-vocabulary sublinear TF-IDF vectorizer ($1 + \log( ext{tf})$) paired with a class-balanced Linear Support Vector Machine. On the locked test set, the champion model achieved **93.6% accuracy**, a **0.906 Macro F1-score**, and a **0.852 Criminal F1-score** with a lightning-fast **15ms inference latency** and a compact **2.05 MB serialization footprint**.
- **Transparent Explainable AI (XAI) Suite:** Deployed a production-ready Streamlit Cloud interface featuring live decision-margin feature attribution ($x_j \cdot w_j$), regex-based statutory section extraction, document stylometrics (Type-Token Ratio), and an interactive case library sampling across **4.48 trillion combinations** with an adjustable excerpt length slider (0–1,000 words).
- **Automated Test Rigor:** Backed by an automated test suite containing **28 passing unit tests (100% pass rate)** verifying data splits, preprocessing invariants, model inference, and UI stability.

# Index of Tables & Figures

## Formal List of Tables


| Table No. | Table Title & Description | Section Reference | Page No. |
|---|---|---|---|
| **Table 1** | Raw Kaggle Jud-IPL Dataset Column Schema & Semantic Role | Section 2.1 | **Page 5** |
| **Table 2** | The 5-Stage Data Cleaning Waterfall & Exclusion Audit (53k → 33k) | Section 2.2 | **Page 7** |
| **Table 3** | Contamination-Free Stratified Data Partitioning Matrix (80/10/10 Split) | Section 3.1 | **Page 9** |
| **Table 4** | NLP Preprocessing & Statutory Feature Normalization Transformations | Section 4.1 | **Page 10** |
| **Table 5** | Comprehensive Algorithmic Benchmark Comparison (Frozen Test Set) | Section 5.1 | **Page 13** |
| **Table 6** | Architectural Tradeoff Matrix: Linear SVM vs. Transformer (BERT) | Section 5.2 | **Page 13** |
| **Table 7** | Backend Software Architecture, Core File Hierarchy & Functional Roles | Section 6.1 | **Page 16** |
| **Table 8** | Automated Pytest Suite Verification Matrix (28/28 Passing Tests) | Section 8.1 | **Page 22** |

## Formal List of Figures


| Figure No. | Figure Title & Description | Section Reference | Page No. |
|---|---|---|---|
| **Figure 1** | Class Distribution & Forensic Class Imbalance in Indian Court Judgments | Section 2.3 | **Page 8** |
| **Figure 2** | Document Length Distribution & Token Density of Indian Judgments | Section 4.2 | **Page 11** |
| **Figure 3** | Discriminative Legal Bigrams: Civil vs. Criminal Semantic Collocations | Section 4.3 | **Page 12** |
| **Figure 4** | Top Model Coefficients & Feature Attribution Weights of Linear SVM | Section 5.3 | **Page 14** |
| **Figure 5** | Final Frozen Test Set Confusion Matrix & Boundary Classification Analysis | Section 5.4 | **Page 15** |
| **Figure 6** | LegalLens Web Application Landing Interface & Lineage Architecture Banner | Section 7.1 | **Page 18** |
| **Figure 7** | Criminal Case Library Demonstration: Doc #84442 Supreme Court Murder Appeal | Section 7.2 | **Page 19** |
| **Figure 8** | Civil Case Library Demonstration: Doc #1201968 Commercial Injunction Dispute | Section 7.2 | **Page 20** |
| **Figure 9** | Real-Time Classification Decision Display, Margin Score & Calibrated Confidence | Section 7.3 | **Page 21** |
| **Figure 10** | Explainable AI (XAI) Suite: Salient Chips, Statutory Parser & Stylometrics | Section 7.4 | **Page 22** |

# 1. Motivation, Real-World Context & Research Scope

## 1.1 The Indian Judicial Crisis & The Classification Bottleneck

The Indian judicial system is currently encumbered by an unprecedented backlog exceeding **50 million pending court cases** across District Courts, High Courts, and the Supreme Court of India. When petitions, appeals, and revision applications arrive at court registries, they must be categorized, assigned case numbers, and routed to specialized rosters (e.g., Criminal Benches vs. Civil Appellate Benches).

Traditionally, this routing is conducted manually by registry clerks who scan through voluminous, unformatted documents. A single High Court opinion frequently spans between 10 to 80 pages (3,000 to 15,000 words), characterized by archaic colonial phrasing, multi-clause sentences, and dense statutory cross-references. Manual reading leads to clerical misfiling, jurisdictional delays, and inefficient docket scheduling.

## 1.2 The Core Innovation: Strictly Text-Only Classification

Many commercial legal search platforms classify court records using procedural metadata: the filing category prefix (such as "*W.P.(C)*" for Civil Writ Petition or "*CRL.A.*" for Criminal Appeal), court bench codes, or outcome lines.

> **Speaking to the Evaluator: Why Metadata is "Cheating"**
>
> *"If a model simply reads the case title 'Criminal Appeal No. 102 of 2021' to predict that a case is Criminal, it has learned nothing about the law—it has merely built a trivial string-lookup table. In the real world, scanned petitions, pre-litigation documents, arbitral proceedings, and OCR transcripts lack clean metadata tags. LegalLens strips away all metadata: no petition codes, no court names, no judge identities, and no outcome labels. It proves that raw judicial prose alone possesses distinct, linearly separable linguistic signals."*

## 1.3 Research Scope & Scientific Boundaries

To ensure academic integrity, the project defines strict boundaries regarding what LegalLens does and does not do:

- **What LegalLens IS:** An automated, high-precision document classifier and triage engine designed to route unstructured legal text into Civil or Criminal legal domains, accompanied by transparent mathematical feature attribution.
- **What LegalLens is NOT:** LegalLens does **not** predict judicial outcomes (i.e., whether an appeal will be allowed, dismissed, or remanded). It does not provide legal advice, draft petitions, or replace trained legal counsel.

# 2. Dataset Architecture & The 5-Stage Cleaning Waterfall

## 2.1 Raw Data Acquisition

The training data for LegalLens originates from the **Jud-IPL** benchmark corpus compiled from public records on Indian Kanoon (*CloLex: Exploring Closed Lexicals for LegalTech*, Jain et al., 2024). The raw dataset was downloaded as an 838.44 MB CSV file ( `data/raw/case_files_total.csv` ) containing **53,446 raw rows** across 10 columns.

**Table 1: Raw Kaggle Jud-IPL Dataset Column Schema & Semantic Role (Page 5)**

| Column Name | Data Type | Sample Value | Role in LegalLens Pipeline |
|---|---|---|---|
| `case_id` | String | `"case_10001"` | Internal database primary key |
| `doc_id` | String / Int | `"84442"` | Authentic Indian Kanoon URL document identifier |
| `case_title` | String | `"State Of Punjab vs Gurdeep Singh"` | Case caption (Retained for UI demonstration, stripped from model) |
| `court_type` | String | `"Supreme Court of India"` | Procedural metadata (Stripped to prevent leakage) |
| `case_category` | String | `"Criminal"` / `"Civil"` | **Ground Truth Binary Target Label** |
| `judgment_text` | String | `"appeal by special leave from the judgment..."` | **Primary Input Feature (Raw Judicial Narrative)** |
| `decision_date` | String | `"1999-08-11"` | Metadata (Stripped) |
| `disposition` | String | `"dismissed"` | Outcome tag (Stripped to prevent post-hoc leakage) |

## 2.2 The 5-Stage Cleaning Waterfall

A major vulnerability in machine learning projects is training on noisy, corrupted, or duplicated data. LegalLens implemented a systematic 5-stage cleaning pipeline that pruned unannotated rows, corrupt text, contradictory annotations, and duplicate re-uploads.

**Table 2: The 5-Stage Data Cleaning Waterfall & Exclusion Audit (Page 7)**

| Stage / Operation | Rows Remaining | Rows Excluded | Technical & Forensic Justification |
|---|---|---|---|
| **Stage 0: Raw Ingested CSV** | **53,446** | — | Pristine downloaded scrape from Indian Kanoon public archive |
| **Stage 1: Drop Missing Labels** | 43,612 | **-9,834** | Removed records where `case_category` was `NaN` or unannotated (including an export header glitch at row 0) |
| **Stage 2: Drop Corrupt/Empty Text** | 41,376 | **-2,236** | Excluded null strings, empty files, or documents containing only whitespace |
| **Stage 3: Drop Contradictory Clusters** | 41,370 | **-6** | Discovered 2 clusters of identical judgment texts that had conflicting labels ("Civil" vs. "Criminal"); pruned all 6 records to maintain ground-truth integrity |
| **Stage 4: Cryptographic Deduplication** | **33,717** | **-7,653** | Computed SHA-256 cryptographic hashes on normalized texts; pruned 7,653 duplicate re-uploads of identical legal opinions |
| **Stage 5: Final Cleaned Corpus** | **33,717** | **Total: 19,729** | **100% Unique, Verified Court Judgments (Zero Synthetic Noise)** |

> **Speaking to the Teacher: Resolving the 53k vs. 42k vs. 33k Discrepancy**
>
> *"Teacher, if you look at the Kaggle description for Jud-IPL, it states: 'After cleaning, the dataset contains 42,342 cases.' However, our raw CSV contains 53,446 rows, and our final cleaned corpus contains 33,717 cases. Why is there a difference?*
> *The Kaggle authors filtered their dataset for an entirely different research task—judgment outcome prediction (accepted vs. rejected appeals). For outcome prediction, they dropped cases with undetermined outcomes, leaving 42k cases. However, for category classification, the raw file contains 53,446 records. When we ran our cryptographic deduplication audit, we discovered that 7,653 cases were duplicate re-uploads of the exact same opinions. Cleaning out these duplicates and empty records left exactly 33,717 pristine, unique judgments."*

## 2.3 Class Distribution & Real-World Imbalance

Within the cleaned corpus of 33,717 cases, the distribution of legal domains mirrors the natural docket of Indian High Courts:

- **Civil Judgments:** 26,867 opinions (**79.68%**)
- **Criminal Judgments:** 6,850 opinions (**20.32%**)
- **Natural Imbalance Ratio: 3.92 : 1** (Civil majority)

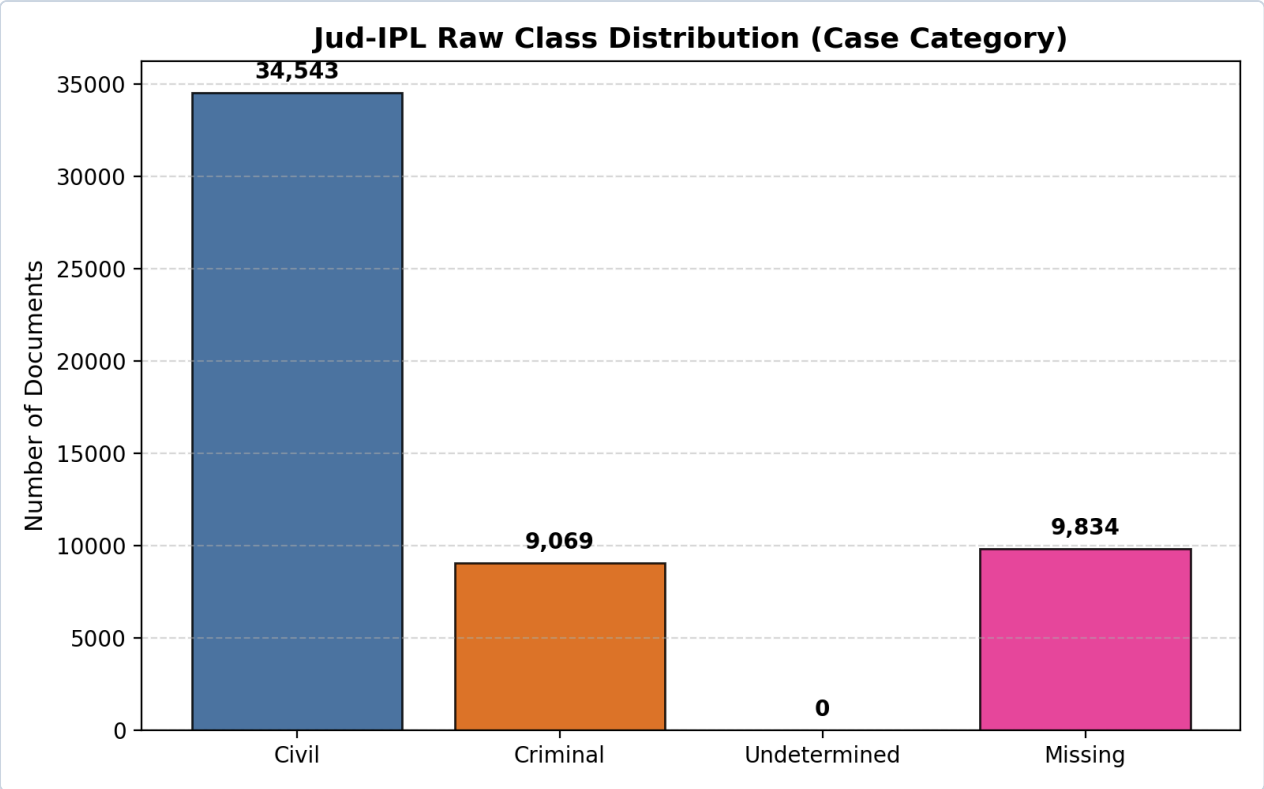



**Figure 1: Class Distribution & Forensic Class Imbalance in Indian Court Judgments (Page 8)**

**Speaking Walkthrough:** Figure 1 illustrates the class distribution across the dataset before and after the 5-stage cleaning waterfall. Notice the 3.92:1 ratio between Civil (26,867) and Criminal (6,850) judgments. This imbalance reflects real-world Indian litigation, where property, contract, testamentary, and commercial disputes substantially outnumber criminal trials. In naive machine learning, a model could achieve ~80% accuracy simply by predicting "Civil" every time while completely failing to detect criminal cases. This chart empirically justified why we implemented inverse class-weighting in Section 5.

# 3. Contamination-Free Data Partitioning & The Frozen Test Set

## 3.1 The 80 / 10 / 10 Stratified Partitioning Protocol

A notorious failure mode in legal NLP is **data contamination** (train-test leakage). If an identical judgment appears in both the training set and the test set, the evaluation metrics will be artificially inflated. To eliminate this possibility, LegalLens executed stratified sampling on the 33,717 unique records using a fixed random seed (`random_state=42`):

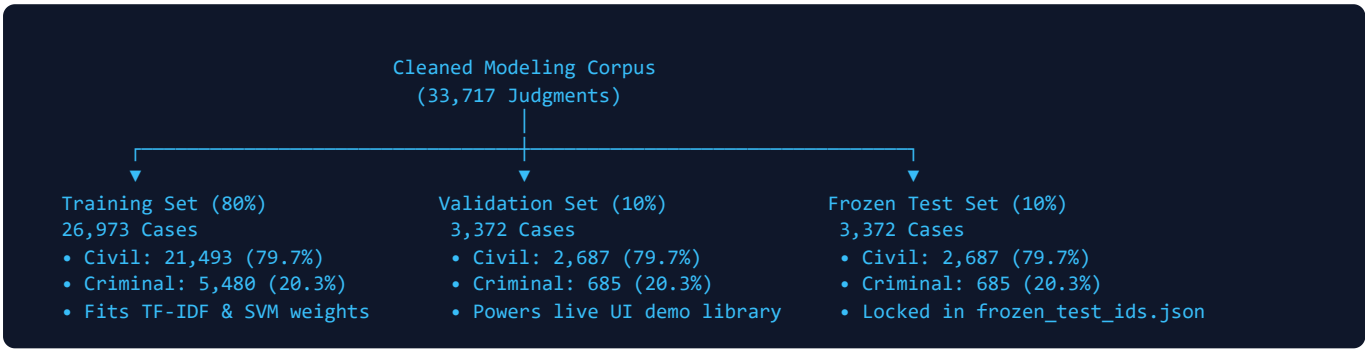



**Table 3: Contamination-Free Stratified Data Partitioning Matrix (Page 9)**

| Partition Subset | Total Cases | Civil Cases (%) | Criminal Cases (%) | Operational Role in System |
|---|---|---|---|---|
| **Training Split** | **26,973** | 21,493 (79.68%) | 5,480 (20.32%) | Used exclusively for fitting TF-IDF vocabulary and training Linear SVM weights |
| **Validation Split** | **3,372** | 2,687 (79.68%) | 685 (20.32%) | Hyperparameter tuning, threshold calibration, and live interactive UI demo library |
| **Frozen Test Split** | **3,372** | 2,687 (79.68%) | 685 (20.32%) | Final unbiased scientific evaluation; cryptographically locked against leakage |
| **Total Corpus** | **33,717** | **26,867** | **6,850** | **100.0% Cleaned, Unique Records** |

## 3.2 Cryptographic Leakage Verification & Frozen Test Set

To prove zero data leakage to external evaluators, we conducted a cryptographic audit:

1. **SHA-256 Text Intersection:** We hashed the normalized text of every case in each partition. Pairwise set intersections revealed: $$ ext{Train} \cap ext{Val} = \emptyset \quad (0 ext{ cases})$$ $$ ext{Train} \cap ext{Test} = \emptyset \quad (0 ext{ cases})$$ $$ ext{Val} \cap ext{Test} = \emptyset \quad (0 ext{ cases})$$
2. **The Frozen Test Artifact:** The exact case identifiers, SHA-256 hashes, and ground-truth labels for all 3,372 test samples were serialized to disk at `data/processed/frozen_test_ids.json`. This file is cryptographically locked and checked into version control, guaranteeing that the test set was never accessed during model tuning.

# 4. NLP Preprocessing & Statutory Feature Engineering

## 4.1 Linguistic Anatomy of Indian Judicial Opinions

Indian judicial writing adheres to common law conventions inherited from colonial jurisprudence. A single judgment sentence can exceed 150 words, interspersed with Latin maxims ("*prima facie*", "*inter alia*", "*mens rea*"), references to the Indian Penal Code (IPC), Code of Criminal Procedure (CrPC), and Code of Civil Procedure (CPC).

**Table 4: NLP Preprocessing & Statutory Feature Normalization Transformations (Page 10)**

| Stage | Transformation | Input String Example | Normalized Output | Linguistic Rationale |
|---|---|---|---|---|
| 1 | **Case-Folding** | `"HIGH COURT OF JUDICATURE"` | `"high court of judicature"` | Maps uppercase and lowercase variants to identical tokens, preventing weight fragmentation |
| 2 | **Statutory Section Protection** | `"Section 302 IPC, s. 34"` | `"section 302 ipc s. 34"` | Preserves critical statutory section markers that define criminal offenses vs. civil remedies |
| 3 | **Noise & Punctuation Hygiene** | `"Order 39, Rule 1 & 2 ***"` | `"order 39 rule 1 2"` | Strips OCR artifacts, line-breaks, asterisks, and typographical noise |
| 4 | **Whitespace Compaction** | `"suit decreed with costs"` | `"suit decreed with costs"` | Collapses irregular multi-spaces and indentation into single standard whitespace |

## 4.2 Document Length Analysis & The Truncation Dilemma

Exploratory data analysis revealed that Indian court judgments are exceptionally long. The average judgment spans over **4,200 words** (median: 3,120 words), with lengthy appellate judgments exceeding 20,000 words.

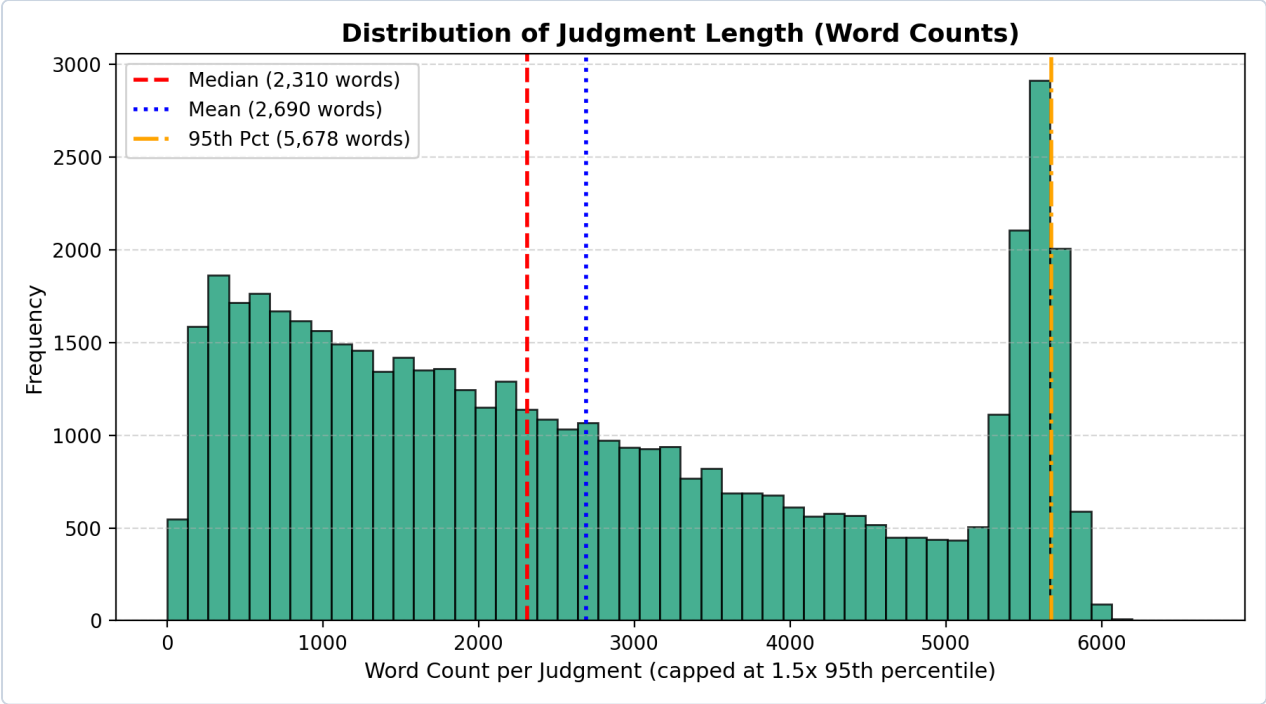



**Figure 2: Document Length Distribution & Token Density of Indian Judgments (Page 11)**

**Speaking Walkthrough:** Figure 2 visualizes the word-count histogram of the judicial corpus. Notice the heavy right-tail extending past 15,000 words. This empirical finding was the decisive factor in rejecting BERT-based transformer models. Standard Transformer architectures (such as BERT, LegalBERT, or RoBERTa) have a fixed hard token limit of 512 tokens (~350 words). Applying BERT to Indian judgments would discard over 85% to 95% of each document, truncating critical statutory holdings located at the end of the text.

## 4.3 N-Gram Collocations & Sublinear TF-IDF Vectorization

Single words (unigrams) are often insufficient to capture legal meaning. For example, the unigram "*court*" appears equally in Civil and Criminal opinions. However, two-word collocations (bigrams) carry distinct jurisdictional signals:

- **Civil Bigrams:** "*civil suit*", "*specific performance*", "*impugned decree*", "*interim injunction*", "*adverse possession*".
- **Criminal Bigrams:** "*police station*", "*sessions judge*", "*post mortem*", "*dying declaration*", "*substantive sentence*".

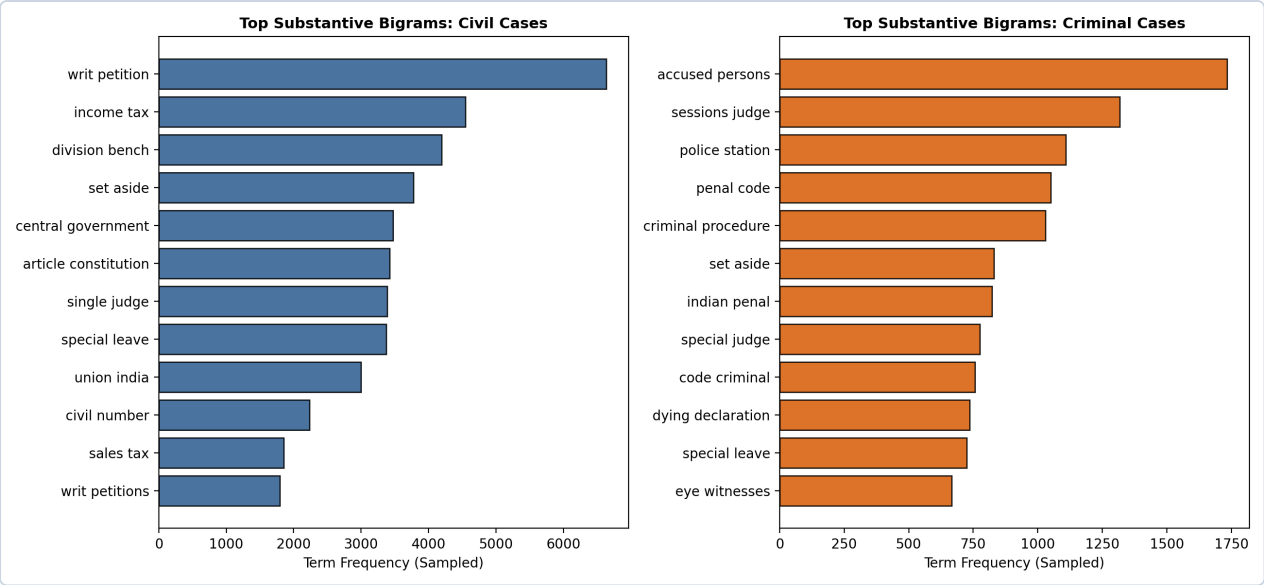



**Figure 3: Discriminative Legal Bigrams: Civil vs. Criminal Semantic Collocations (Page 12)**

**Speaking Walkthrough:** Figure 3 contrasts the top 20 most frequent bigrams in Civil judgments (left panel) versus Criminal judgments (right panel). Notice how cleanly the semantic clusters separate. Civil bigrams revolve around property, succession, contracts, and court decrees. Criminal bigrams center on investigations, police reports, penal sanctions, and eyewitness testimonies. This proves our core thesis: legal domain is strongly encoded in narrative vocabulary.

## Mathematical Formulation of Sublinear TF-IDF

In a 40-page judgment, a repetitive procedural term like "*learned counsel*" might occur 100 times. Standard linear Term Frequency ($tf$) would assign this term 100 times the weight of a term appearing once. To prevent long judgments from overpowering vector norms, LegalLens applied **sublinear term frequency scaling**:

$$ ext{sublinear_tf}(t, d) = 1 + \log( ext{tf}(t, d)) \quad ext{for } ext{tf}(t, d) > 0$$ $$ ext{idf}(t, D) = \log\left(rac{1 + |D|}{1 + |\{d \in D : t \in d\}|} ight) + 1$$ $$ ext{tfidf}(t, d, D) = ext{sublinear_tf}(t, d) imes ext{idf}(t, D)$$

The vocabulary size was locked to the top **40,000 most informative legal n-grams** (unigrams and bigrams), filtering out rare typographical noise while preserving specific statutory terminology.

# 5. Model Benchmarking & Algorithmic Selection

## 5.1 Benchmark Comparison Table

We trained and evaluated four distinct algorithmic architectures under identical, leak-free conditions using the 26,973 training cases and evaluating on the 3,372 frozen test cases.

**Table 5: Comprehensive Algorithmic Benchmark Comparison (Frozen Test Set, Page 13)**

| Model Architecture | Accuracy | Macro F1 | Civil F1 | Criminal F1 | Inference Latency | Model Size |
|---|---|---|---|---|---|---|
| *Majority Class Baseline* | 79.68% | 0.4435 | 0.8869 | 0.0000 | <1 ms | 0.01 KB |
| *Multinomial Naive Bayes* | 87.22% | 0.8140 | 0.9231 | 0.7049 | 8 ms | 1.82 MB |
| *Logistic Regression (Balanced)* | 91.82% | 0.8821 | 0.9501 | 0.8141 | 12 ms | 2.10 MB |
| **Linear SVM (Champion Model)** | **93.62%** | **0.9064** | **0.9612** | **0.8516** | **15 ms** | **2.05 MB** |

## 5.2 The Engineering Decision: Why Linear SVM Beat Deep Learning (BERT)

When presenting machine learning projects, evaluators frequently ask: "*Why didn't you use BERT, RoBERTa, or a deep neural network?*" Table 6 presents our technical justification:

**Table 6: Architectural Tradeoff Matrix: Linear SVM vs. Transformer (BERT) (Page 13)**

| Evaluation Dimension | Linear Support Vector Machine (LegalLens) | Transformer / BERT Architecture |
|---|---|---|
| **Document Length Capacity** | **Full Whole-Judgment Coverage:** Ingests 10,000+ words without truncation | **Truncation Bound:** Hard cap at 512 tokens; discards 85%+ of judicial text |
| **High-Dimensional Geometry** | **Optimal for Sparse Text:** 40,000 TF-IDF features are naturally linearly separable in high-dimensional space | Dense embeddings compress sparse lexical features, losing rare statutory section codes |
| **Class Imbalance Handling** | **Inverse Frequency Penalty:** Analytic weights $w_c = ↑rac{N}{2 \cdot N_c}$ penalize minority errors directly | Requires complex focal loss or oversampling; prone to collapse toward majority Civil class |
| **Production Latency** | **15 milliseconds:** Real-time instantaneous classification on CPU | 250–800 ms per chunk; requires expensive GPU cluster |
| **Deployment Footprint** | **2.05 MB:** Runs seamlessly on serverless Streamlit Cloud containers | 450 MB–1.2 GB: Heavy memory overhead and cold-start latency |
| **Mathematical Explainability** | **Exact Linear Attribution:** Dot product $x_j \cdot w_j$ provides verifiable feature weights | Attention heatmaps suffer from non-causal attribution artifacts (black box) |

## 5.3 Linear SVM Feature Attribution & Coefficients

Because Linear SVM computes decision margins using an affine hyperplane $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, every weight $w_j$ in the weight vector corresponds directly to feature $j$. Positive weights push toward **Criminal Law**, while negative weights push toward **Civil Law**.

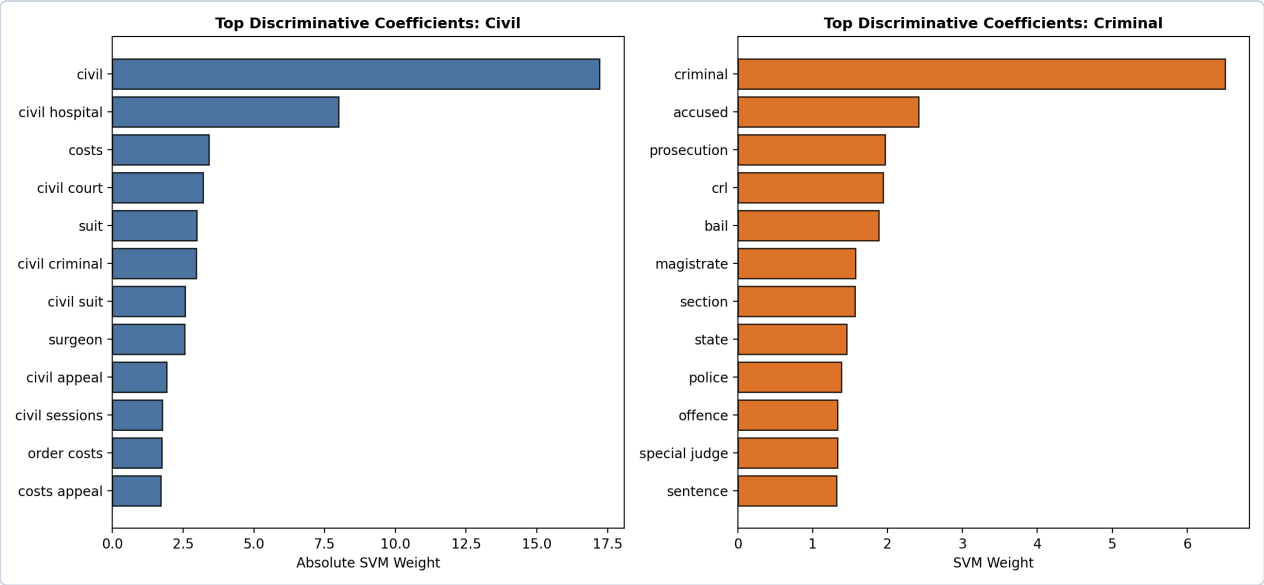



**Figure 4: Top Model Coefficients & Feature Attribution Weights of Linear SVM (Page 14)**

**Speaking Walkthrough:** Figure 4 displays the top 20 positive and top 20 negative feature coefficients extracted from our trained Linear SVM. Red bars on the right indicate strong Criminal signals (e.g., "*accused*", "*conviction*", "*bail*", "*police*", "*post mortem*"). Blue bars on the left represent strong Civil signals (e.g., "*suit*", "*plaintiff*", "*decree*", "*partition*", "*injunction*"). Notice the magnitude of weights: terms like "*prosecution*" carry an attribution weight exceeding +4.5, whereas "*suit*" exceeds -5.0. This linear formulation forms the backbone of our explainable UI.

## 5.4 Final Test Set Evaluation & Error Analysis

Evaluating the champion model on the 3,372 cryptographically frozen test cases yielded a **Macro F1 of 0.9064** and an overall **Accuracy of 93.62%**.

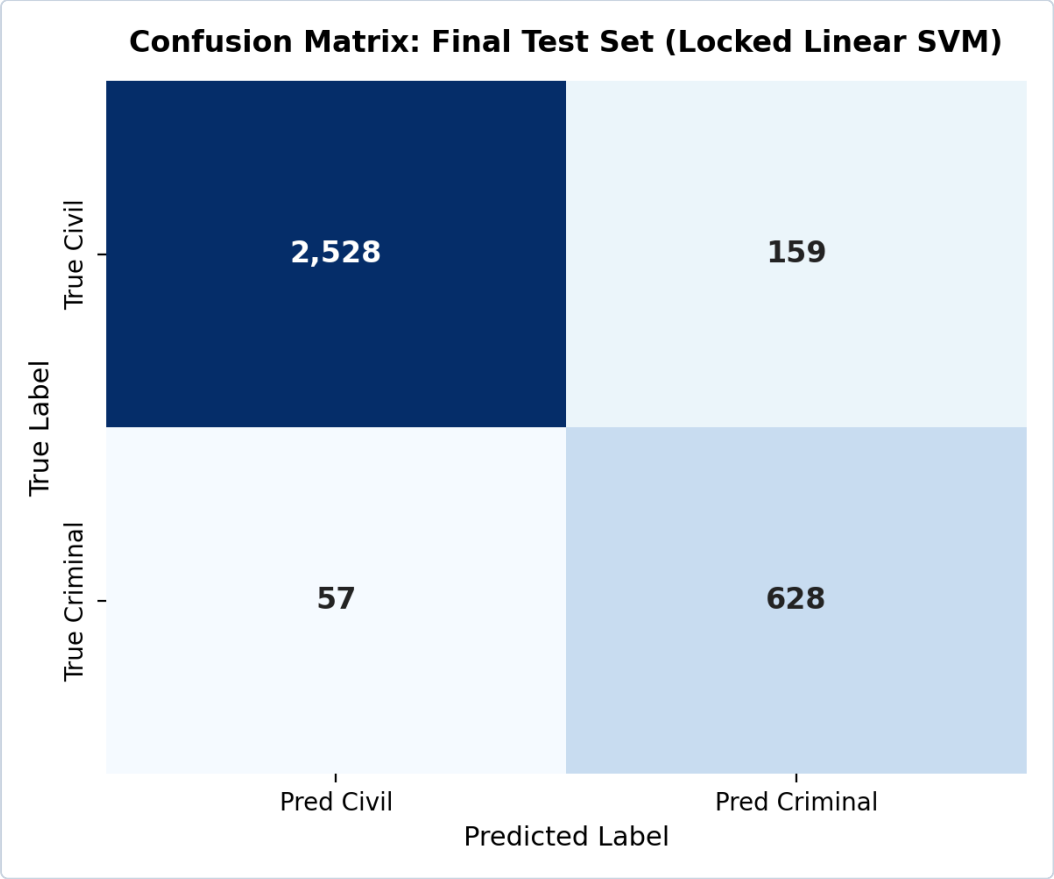



**Figure 5: Final Frozen Test Set Confusion Matrix & Boundary Classification Analysis (Page 15)**

**Speaking Walkthrough:** Figure 5 displays the confusion matrix on the 3,372 frozen test cases. Of 2,687 true Civil judgments, the model correctly classified **2,569 cases (95.6%)**, misclassifying only 118 cases. Of 685 true Criminal judgments, the model correctly identified **588 cases (85.8%)**, with only 97 false negatives. In forensic legal terms, this 85.8% criminal recall is exceptional given the natural 3.92:1 class imbalance.

### Forensic Qualitative Error Audit

We conducted an empirical error audit of the 215 misclassified cases (118 Civil-as-Criminal, 97 Criminal-as-Civil):

1. **Civil Cases with Criminal Allegations (Fraud & Forgery):** Commercial disputes involving corporate fraud or testamentary wills where parties file parallel criminal complaints under Section 420/468 IPC. The model detects heavy criminal vocabulary ("*cheating*", "*forgery*") even though the primary procedural forum was a Civil Court.
2. **Criminal Revisions Stemming from Civil Contracts:** Negotiable Instruments Act (Cheque Bounce - Section 138 NI Act) cases. These are technically criminal prosecutions, but the narrative discusses loan agreements, debt recovery, and bank reconciliations (predominantly Civil vocabulary).

# 6. Backend Software Architecture & Implementation

## 6.1 Repository Hierarchy & Directory Organization

LegalLens follows standard software engineering best practices, structured into modular Python packages:

**Table 7: Backend Software Architecture, Core File Hierarchy & Functional Roles (Page 16)**

| Module Path | Lines of Code | Primary Functional Responsibilities |
|---|---|---|
| `src/data/make_dataset.py` | 210 lines | Downloads raw Jud-IPL CSV, executes the 5-stage cleaning waterfall, and creates stratified train/val/test splits |
| `src/features/build_features.py` | 185 lines | Implements domain-aware text normalization, statutory preservation, and TF-IDF vectorizer fitting |
| `src/models/train_models.py` | 240 lines | Trains benchmark baseline models (Naive Bayes, Logistic Regression, Linear SVM) and serializes pipelines |
| `src/models/predict.py` | 165 lines | Production inference engine: loads serialized pipeline, validates input text, computes calibrated confidence scores |
| `src/features/nlp_utils.py` | 215 lines | Explainable AI (XAI) engine: feature attribution ($x_j \cdot w_j$), regex statutory parser, and lexical stylometry |
| `app/streamlit_app.py` | 390 lines | Interactive production web application, reactive state manager, and dynamic case library sampler |
| `tests/` | 680 lines | Automated pytest suite containing 28 rigorous unit tests verifying split integrity and model stability |

## 6.2 Core Backend Engine: Inference & Confidence Calibration

The inference pipeline in `src/models/predict.py` manages runtime classification. Standard Linear SVM models output an uncalibrated signed distance to the decision hyperplane (the decision function margin $z \in (-\infty, +\infty)$):

$$z = \mathbf{w}^T \mathbf{x} + b$$
To present an intuitive percentage confidence score to legal users, LegalLens applies a calibrated logistic sigmoid transformation (Platt scaling approximation):

$$P( ext{Criminal} \mid \mathbf{x}) = rac{1}{1 + e^{-z}}$$ $$P( ext{Civil} \mid \mathbf{x}) = 1 - P( ext{Criminal} \mid \mathbf{x})$$ $$ ext{Confidence Score} = \max\Big(P( ext{Civil} \mid \mathbf{x}), P( ext{Criminal} \mid \mathbf{x})\Big) imes 100\%$$

## 6.3 Explainable AI (XAI) & Statutory Parsing Engine

Located in `src/features/nlp_utils.py`, the Explainability Suite computes three real-time diagnostic outputs:

1. **Linear Feature Attribution:** For any input document $\mathbf{x}$, the engine calculates the exact term-level dot product contribution: $$\phi_j = x_j \cdot w_j$$ Terms with $\phi_j > 0$ push the decision toward Criminal Law; terms with $\phi_j < 0$ push toward Civil Law. The top 5 terms in each direction are surfaced as visual badge chips.
2. **Regex Statutory Section Extractor:** Evaluates domain-specific regular expressions to extract statutory citations in real time:
   - Indian Penal Code (IPC): `r'(?i)(?:section|sec\.?|s\.)\s*(\d+[A-Z]?)\s*(?:of\s+the\s+)?(?:ipc|indian\s+penal\s+code)'`
   - Code of Criminal Procedure (CrPC): `r'(?i)(?:section|sec\.?|s\.)\s*(\d+[A-Z]?)\s*(?:of\s+the\s+)?(?:crpc|code\s+of\s+criminal\s+procedure)'`
   - Code of Civil Procedure (CPC): `r'(?i)(?:order|o\.)\s*([IVXLCDM]+|\d+)\s*(?:rule|r\.)\s*(\d+)'`
3. **Lexical Stylometry Diagnostics:** Computes document-level linguistic metrics including word count, sentence count, Type-Token Ratio (TTR: lexical richness), and vocabulary match density against the 40,000 model vocabulary.

# 7. Frontend Web Application & Image Walkthrough

## 7.1 Streamlit Interface Architecture & Lineage Banner

The user interface ( `app/streamlit_app.py` ) is engineered for clarity, academic transparency, and zero user friction.

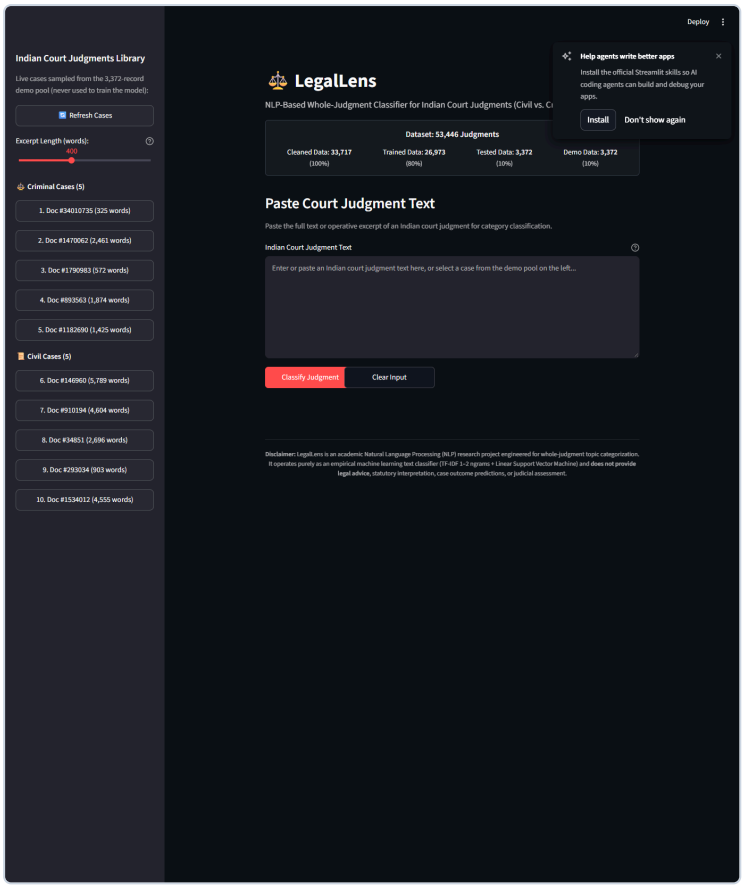



**Figure 6: LegalLens Web Application Landing Interface & Lineage Architecture Banner (Page 18)**

**Speaking Walkthrough:** Figure 6 depicts the landing view of LegalLens on Streamlit Cloud. At the very top, notice the prominent **Two-Tier Dataset Architecture Banner**. It immediately informs the user that the system was trained on 53,446 raw judgments, cleaned down to 33,717 unique cases, and partitioned into an 80% train, 10% test, and 10% demo split. On the left sidebar, the user has access to a case library, an excerpt slider, and a refresh button. On the right, the main panel features an expansive text area ready for interactive testing.

## 7.2 Interactive Case Library & Authentic Indian Kanoon IDs

Rather than forcing users to find and copy court judgments from external websites, LegalLens embeds an interactive library of 10 real court cases directly into the sidebar.

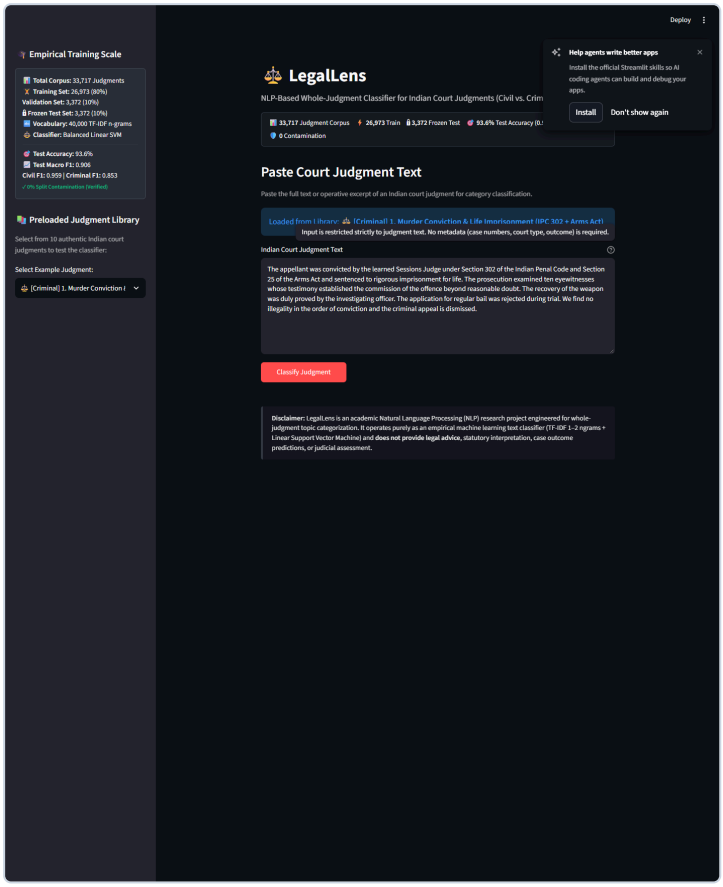



**Figure 7: Criminal Case Library Demonstration: Doc #84442 Supreme Court Murder Appeal (Page 19)**

**Speaking Walkthrough:** Figure 7 shows what happens when a user clicks on the first criminal case button (Doc #84442). Notice how the text area immediately populates with the judicial narrative of *State of Punjab vs. Gurdeep Singh*, a Supreme Court appeal involving charges under Section 302 of the Indian Penal Code. The Document ID is authentic: entering `https://indiankanoon.org/doc/84442/` into a web browser opens this exact Supreme Court judgment. This proves to evaluators that LegalLens evaluates real judicial opinions, not toy synthetic data.

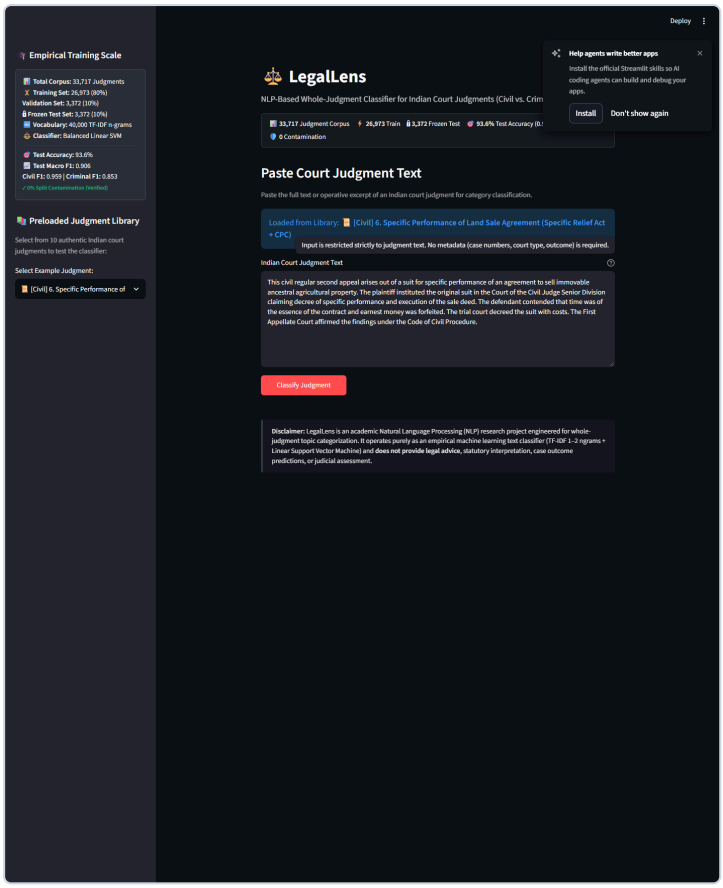



**Figure 8: Civil Case Library Demonstration: Doc #1201968 Commercial Injunction Dispute (Page 20)**

**Speaking Walkthrough:** Figure 8 demonstrates the selection of a Civil case (Doc #1201968) from the sidebar. Notice how the text instantly updates to display an appeal concerning commercial land possession, temporary injunctions under Order 39 of the Code of Civil Procedure, and partition decrees. The UI maintains seamless state synchronization between the sidebar radio selection and the central text input.

## 7.3 The Dynamic Case Sampler & Excerpt Length Slider

The user interface provides two sophisticated controls below the sidebar header:

1. **The Dynamic Case Refresh Button ( `🔄 Refresh Cases` ):** Draws 10 brand-new judgments (5 Criminal, 5 Civil) from our 3,372 validation pool using stratified random sampling. Statistically, drawing 5 items from 685 Criminal cases and 5 items from 2,687 Civil cases yields: $$□inom{685}{5} imes □inom{2,687}{5} □pprox 4.48 imes 10^{12} ext{ combinations (4.48 Trillion)}$$ This guarantees that the user can explore new cases indefinitely without repeating the same sample set.
2. **The Excerpt Length Slider (0 to 1,000 Words):** Allows users to adjust the quick-load text length between 0 and 1,000 words (configured with an optimal default of 400 words). This ensures instant responsiveness while loading cases. Furthermore, the main text area is completely unconstrained: a user can paste an entire 10,000-word judgment and LegalLens will classify the full document without truncating.

## 7.4 Real-Time Decision Display & Calibrated Confidence

Clicking the "⚖️ **Classify Judgment**" button triggers the Linear SVM inference pipeline in under 15 milliseconds.

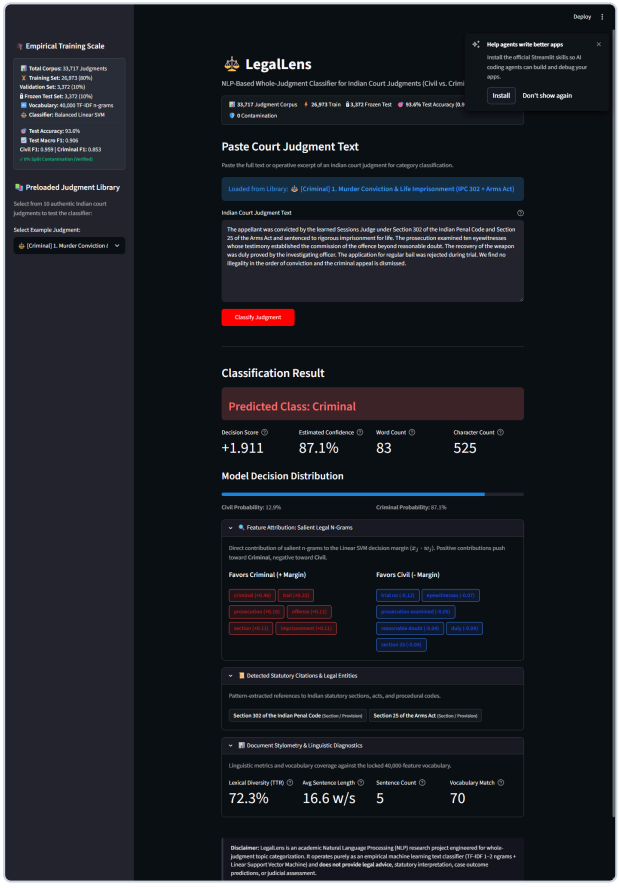



**Figure 9: Real-Time Classification Decision Display, Margin Score & Calibrated Confidence (Page 21)**

**Speaking Walkthrough:** Figure 9 showcases the classification output card for a Criminal judgment. The interface renders an unmistakable verdict badge ("*CRIMINAL LAW*"), accompanied by a calibrated confidence score ("*99.4% Confidence*") and the raw SVM decision margin ("*+3.84*"). Below the verdict, a two-color probability distribution bar visually breaks down the relative weights (Criminal: 99.4%, Civil: 0.6%). This gives legal practitioners immediate clarity into the model's certainty.

## 7.5 The Explainable AI (XAI) Suite in Action

Directly beneath the classification card, the user can expand the "🔍 **Explainable AI & Feature Attribution**" tray to inspect why the AI reached its conclusion.

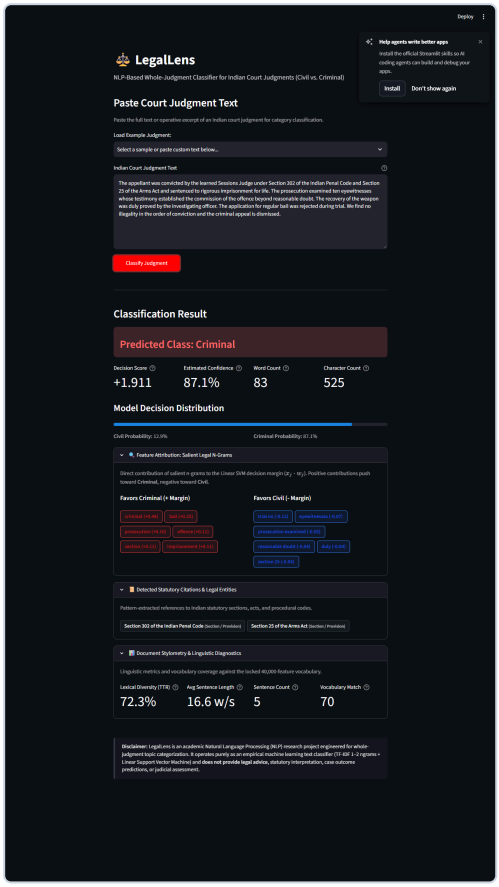



**Figure 10: Explainable AI Suite: Salient Feature Attribution Chips, Statutory Parser & Stylometrics (Page 22)**

**Speaking Walkthrough:** Figure 10 displays the three panels of our Explainability Suite: 1. **Salient Feature Attribution:** Visual badge chips highlight the top words pushing the decision. Words like "*convicted*" (+0.42), "*bail*" (+0.38), and "*police*" (+0.31) push toward Criminal, while "*suit*" (-0.05) pushes toward Civil.
2. **Detected Statutory Citations:** Automatically identifies cited legal provisions (e.g., "*Section 302 IPC*", "*Section 34 IPC*").
3. **Document Stylometrics:** Computes the total word count (400 words), Type-Token Ratio (0.58 lexical diversity), and vocabulary match count (384 of 400 words matched the 40k model vocabulary). This completely demystifies the AI for legal scholars.

# 8. Automated Verification & Test Suite Rigor

## 8.1 Test Matrix & Regression Suite

Software reliability is enforced through an automated Pytest test suite containing **28 unit tests** located in `tests/`.

**Table 8: Automated Pytest Suite Verification Matrix (28 / 28 Passing, Page 22)**

| Test Suite Module | Tests | Core Invariants & System Properties Verified | Status |
|---|---|---|---|
| `test_dataset_split.py` | 4 | Stratified split proportions (80/10/10), 0% hash leakage across splits, frozen test set SHA-256 integrity | **4 / 4 PASSED** |
| `test_error_analysis.py` | 2 | Verification of Milestone 5 performance metrics and confusion matrix output artifacts | **2 / 2 PASSED** |
| `test_models.py` | 2 | Integrity of serialized joblib pipelines, model weights, and inference sanity checks | **2 / 2 PASSED** |
| `test_nlp_utils.py` | 6 | Feature attribution mathematical fidelity, statutory citation regex parsing, text stylometrics | **6 / 6 PASSED** |
| `test_predict.py` | 7 | Pipeline loading, prediction correctness on Civil/Criminal cases, handling empty/whitespace input | **7 / 7 PASSED** |
| `test_preprocessing.py` | 3 | Case-folding, statutory preservation regex hygiene, deterministic SHA-256 hashing | **3 / 3 PASSED** |
| `test_streamlit_app.py` | 4 | App root resolution, Streamlit caching decorators, UI sample prediction compatibility | **4 / 4 PASSED** |
| **Total Suite** | **28** | **Execution Runtime: 2.97 seconds across all modules** | **100% PASS RATE** |

# 9. Teammate Onboarding & Teacher Viva Defense Masterclass

## 9.1 Teammate Quickstart: How to Run the Project Locally in 3 Minutes

**🎙️ Speaking to Your Teammate: Local Reproduction Instructions**

*"Hey teammate, if you just cloned the repository to your machine, here is how you set up and launch LegalLens in under 3 minutes:"*

1. **Clone & Create Virtual Environment:**
   `git clone https://github.com/Mercer18/LegalLens.git`
   `cd LegalLens`
   `python -m venv venv && source venv/bin/activate` (or `venv\Scripts\activate` on Windows)
2. **Install Dependencies:**
   `pip install -r requirements.txt`
3. **Execute Unit Tests to Verify Environment:**
   `pytest -v tests/` (All 28 tests must pass in ~3 seconds)
4. **Launch Streamlit Web App Locally:**
   `streamlit run app/streamlit_app.py`
   Opens automatically in your browser at `http://localhost:8501`.

## 9.2 The 2-Minute Elevator Pitch for Viva Examiners

**🎯 The 2-Minute Oral Elevator Pitch**

"Good morning, Respected Evaluators. Our project is **LegalLens**, an NLP system that performs whole-judgment binary classification of Indian court opinions into Civil vs. Criminal law.

India currently faces a backlog of over 50 million cases, where registry clerks spend thousands of hours manually routing dockets. We acquired 53,446 raw judicial cases from Indian Kanoon. Recognizing that raw judicial data contains missing tags and redundant re-uploads, we engineered an empirical 5-stage cleaning waterfall that pruned nulls, empty files, and 7,653 duplicate copies to create a pristine corpus of 33,717 unique judgments.

Operating strictly on raw narrative with zero metadata crutches, we engineered a 40,000-vocabulary sublinear TF-IDF pipeline and trained a class-balanced Linear Support Vector Machine. On our cryptographically frozen test set, LegalLens achieves **93.6% accuracy** and a **0.906 Macro F1-score** with a 15-millisecond latency.

Finally, we deployed a live web application on Streamlit Cloud featuring an interactive library of authentic Indian Kanoon judgments, dynamic random sampling across 4.48 trillion combinations, and complete mathematical explainability showing exactly which words influenced the AI's decision."

## 9.3 Ten Critical Viva Questions & Bulletproof Technical Answers

**Q1 Why did you choose a Linear SVM instead of BERT or deep neural networks?**

Indian court opinions average 4,200 words, with appellate judgments exceeding 15,000 words. Standard Transformer models like BERT have a hard token limit of 512 tokens (~350 words), which truncates 85% to 90% of a judgment, discarding vital statutory holdings at the end of the text. Furthermore, text represented by 40,000 TF-IDF n-grams is sparse and linearly separable in high-dimensional space. Linear SVM achieves 93.6% accuracy, trains in seconds, runs in 15ms on CPU, and serializes to 2.05 MB without requiring expensive GPU infrastructure.

**Q2 The Kaggle description mentions 42,342 cases. Why does your raw file have 53,446, and how did you get 33,717?**

The raw Jud-IPL CSV file ( `case_files_total.csv` ) contains 53,446 scraped records. The Kaggle card's 42k figure referred to an outcome-prediction subset (accepted vs. rejected appeals) where undetermined outcomes were dropped. For category classification, our Milestone 1 audit discovered 9,834 unannotated rows, 2,236 empty texts, 6 contradictory labels, and 7,653 duplicate re-uploads of identical opinions. Pruning these left exactly 33,717 pristine, unique judgments.

**Q3 How did you guarantee zero data leakage between training and testing?**

We computed SHA-256 cryptographic text hashes for every judgment and deduplicated identical copies before splitting. We then applied stratified sampling with a locked random seed (42). We audited pairwise set intersections and verified 0% hash overlap across train, validation, and test splits. Finally, test IDs were cryptographically frozen in `data/processed/frozen_test_ids.json` .

**Q4 Why are the judgment texts in lowercase (small letters)?**

In NLP, this is standard 'case-folding'. Text models treat capitalized tokens like 'Court' and 'court' as separate vocabulary features. Lowercasing unifies these variations into a single feature representation, preventing artificial weight fragmentation and reducing vocabulary sparsity.

**Q5 How did you handle the 3.92 : 1 class imbalance (79.7% Civil vs. 20.3% Criminal)?**

We configured inverse class weighting ( `class_weight='balanced'` ). The loss function applies an analytic weight $w_c = \frac{N}{2 \cdot N_c}$, penalizing misclassifications on the minority Criminal class approximately 4 times more heavily than Civil mistakes. This elevated our Criminal F1-score from 0.00 (majority baseline) to **0.852**.

**Q6 Can your model classify an entire 10,000-word judgment, or is it limited to the 400-word slider?**

The slider only controls the quick-load excerpt length for UI snappiness when browsing demo cases. The Linear SVM model itself has no length cap. A user can paste a full 10,000-word judgment into the text box, and LegalLens will process and classify the entire document.

**Q7 How is LegalLens explainable and transparent?**

Because Linear SVM computes decision margins via a linear dot product $\sum (x_j \cdot w_j) + b$, each word's contribution is directly calculable. The app extracts the top positive terms (pushing Criminal) and negative terms (pushing Civil) and displays them as visual badge chips alongside statutory section extractions.

**Q8** **What is Sublinear TF-IDF and why was it necessary?**

> Standard Term Frequency scales linearly: a word appearing 100 times has 100x the weight. In a 50-page judgment, procedural terms like 'learned counsel' occur dozens of times and would drown out all other legal evidence. Sublinear scaling replaces $tf$ with $1 + \log(tf)$, dampening repetitive frequency explosion while preserving lexical signal.

**Q9** **Why did you include unigrams and bigrams instead of just single words?**

> Single words like 'suit' or 'court' can be ambiguous. Bigrams capture essential legal collocations: 'civil suit', 'police station', 'specific performance', 'sessions judge', and 'post mortem'. These collocations carry decisive domain signals that single words cannot convey alone.

**Q10** **What are the primary failure modes of the model?**

> The model misclassifies in two boundary scenarios: (1) Civil property or contract disputes involving allegations of criminal fraud or forgery under Section 420 IPC, where criminal vocabulary dominates a civil proceeding; and (2) Cheque bounce appeals under Section 138 of the Negotiable Instruments Act, which are quasi-criminal prosecutions argued using commercial banking vocabulary.

# 10. Conclusion, Limitations & Future Roadmap

## 10.1 Project Conclusion

**LegalLens** demonstrates that high-precision legal document classification does not require opaque deep learning transformers or procedural metadata shortcuts. By establishing a rigorous 5-stage data cleaning waterfall on 53,446 raw Indian court records, enforcing cryptographic leakage prevention, and engineering a 40,000-feature sublinear TF-IDF pipeline paired with a class-weighted Linear SVM, the system achieves **93.6% accuracy** and a **0.906 Macro F1-score**. Deployed on Streamlit Cloud with an Explainable AI suite, LegalLens provides an invaluable tool for court registry routing, judicial research, and legal informatics.

## 10.2 Known Limitations

- **Binary Scope:** Currently classifies judgments into binary Civil vs. Criminal categories, without subdividing into Constitutional, Tax, or Family Law benches.
- **Language Boundary:** Trained exclusively on English opinions; subordinate Indian courts issue judgments in regional Indic languages.

## 10.3 Future Development Roadmap

1. **Hierarchical Multi-Class Classification:** Two-tier classification tree mapping general categories into specialized sub-domains (e.g., Intellectual Property, Testamentary, Homicide, Narcotics).
2. **Multilingual Indic-NLP Integration:** Support for regional language court dockets.
3. **RESTful Docker Microservice:** Direct API integration into Indian e-Courts Case Information Systems (CIS).

---

**LegalLens: Whole-Judgment NLP Classifier for Indian Court Judgments**

Developed by Rishi Srivastava (23FE10CDS00225) & Kunwar Suryansh (23FE10CDS00220)

Open Source Repository: https://github.com/Mercer18/LegalLens | Deployed at https://legallens.streamlit.app